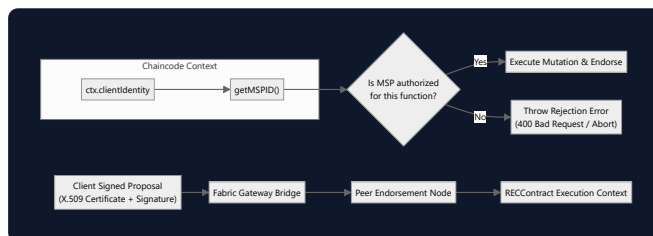


Security & Access Control Specification

This document details the security architecture, identity federation, cryptographic enforcement, and threat mitigation mechanisms of the REC Fraud Detection & Tracking System.

1. Fabric MSP Identity & Authorization

In enterprise distributed ledgers, access control **must never rely solely on frontend or API checks**. The smart contract execution environment itself validates identity credentials.



1.1 Organizational Authorization Matrix

Chaincode Function	Authorized MSP(s)	Unauthorized MSPs	Enforcement Mechanism
<code>createREC</code>	<code>IssuerOrgMSP</code>	<code>BuyerOrgMSP</code> , <code>AuditorOrgMSP</code> , <code>RegulatorOrgMSP</code>	In-contract check: <code>ctx.clientIdentity.getMSPID() === 'IssuerOrgMSP'</code>
<code>transferREC</code>	Authorized Account Holder	Non-holders, Unapproved Third Parties	In-contract balance check: <code>balances[fromOwner] ≥ quantity</code>
<code>retireREC</code>	Authorized Unit Owner	Non-owners, Third Parties	In-contract balance check: <code>balances[owner] ≥ quantity</code>
<code>cancelREC</code>	<code>RegulatorOrgMSP</code> , <code>IssuerOrgMSP</code>	<code>BuyerOrgMSP</code> , <code>AuditorOrgMSP</code>	In-contract check: <code>getMSPID() in ['RegulatorOrgMSP', 'IssuerOrgMSP']</code>
<code>getREC</code>	All 4 Organizations	Non-network entities	Channel read access policy
<code>getRECHistory</code>	All 4 Organizations	Non-network entities	Channel read access policy
<code>verifyREC</code>	All 4 Organizations	Non-network entities	Channel read access policy

2. Cryptographic Document Protection

2.1 Why Documents Stay Off-Chain

- Storing large binary files (PDFs, TIFF scans) on blockchain causes state bloat, degrades peer gossip latency, and violates privacy regulations (such as GDPR or commercial confidentiality).
- The REC system computes a **SHA-256 cryptographic digest** of the raw document file bytes:
$$\text{documentHash} = \text{SHA256}(\text{document_bytes})$$
- This 64-character hexadecimal string is written directly to the ledger during `createREC`.

2.2 Tamper Detection Mechanism

- When any party re-uploads a document for verification:
- The backend recalculates $\text{SHA256}(\text{uploaded_bytes})$.
- The backend retrieves the original `documentHash` committed to Hyperledger Fabric.
- If even a single byte has changed, the hash diverges completely (avalanche effect).
- The system flags `DOCUMENT INTEGRITY FAILURE`.

3. Threat Modeling & Countermeasures

Threat Scenario	Attack Vector	Countermeasure
Double Issuance	Generator attempts to mint RECs twice for the same vintage.	Off-chain DB uniqueness check + on-chain <code>recId</code> existence check + duplicate document hash check in fraud engine.
Over-Transfer	Buyer attempts to sell more RECs than held in inventory.	Smart contract balance enforcement rejects with <code>INSUFFICIENT REC BALANCE</code> .
Resale of Retired RECs	Participant attempts to transfer units that were retired for Scope 2 offset.	Retired units decrement <code>activeQuantity</code> and are credited to <code>retiredQuantity</code> . Smart contracts strictly restrict transfer to <code>activeQuantity</code> .
Revocation Bypass	Trader attempts to liquidate a cancelled certificate.	Smart contract verifies <code>status !== "CANCELLED"</code> prior to allowing any transfer.
Rogue Issuance	Compromised buyer key attempts to mint synthetic certificates.	Smart contract asserts caller MSP is <code>IssuerOrgMSP</code> ; any proposal from <code>BuyerOrgMSP</code> is rejected during peer endorsement.
Data Alteration	Central database admin attempts to modify past certificate ownership.	Fabric transactions are cryptographically chained in blocks signed by the ordering service and all endorsing peers; historical modifications are impossible.

4. Secrets & Credential Management Policy

- Zero Hardcoded Secrets:** All configuration values are injected via environment variables (`.env`).
- Exclusion from Version Control:** `.gitignore` strictly excludes:
- Private keys (`*.key`, `keystore/`)
- Digital certificates (`*.pem`, `*.crt`)

5. Uploaded documents (`uploads/`)

6. SQLite databases (`*.db`)

7. **Template Provided:** `.env.example` provides explicit setup parameters without leaking sensitive operational credentials.